

Agentic GraphRAG for Supply Chain Risk Assessment

1-Week Execution Plan — 2-Person Team

[A] Person A [B] Person B [A+B] Both [optional] Time-permitting extension

Golden rule: Build one impressive end-to-end demo that works reliably. A shallow but complete pipeline (data → graph → agent → risk output) beats a half-built sophisticated one every time.

Scope: EV battery supply chain. Focal companies: **Tesla, CATL, Panasonic, LG Energy Solution**. Critical minerals: **lithium, cobalt, nickel**. Key geographies: **DRC, Chile, Australia, China**.

Extension scope (if time allows): After the core pipeline is stable, add a private data integration layer that allows a company to ingest its own internal supplier data and merge it with the public graph for enriched, customized risk assessment.

Tech Stack

- **Neo4j Aura** (free tier) — graph database, cloud-hosted, no local setup
 - **Python 3.11+** — all scripting and pipelines
 - **LangGraph** — agentic orchestration (ReAct loop)
 - **OpenAI GPT-4o or Claude API** — LLM for extraction + generation
 - **Tavily API** — web search tool for the agent (free tier)
- **NetworkX + Matplotlib** — graph visualization

Avoid: Docker, microservices, vector databases (unless needed), complex infrastructure. A Jupyter notebook for extraction and a Python script for the agent is enough.

Day 1 — Setup & Data Collection

[A+B]

Environment Setup

1. Create Neo4j Aura free account at neo4j.com/cloud/aura. Save connection URI, username, password.
2. Install Python dependencies:

```
pip install langchain langgraph
pip install langchain-openai neo4j
pip install tavily-python networkx
pip install matplotlib pandas requests
```
3. Get API keys: OpenAI (or Anthropic), Tavily, Neo4j.
4. Create a shared `.env` file with all credentials.
5. Test Neo4j connection with a simple `CREATE` node query.

Data Collection (both collect in parallel)

Target: **25–35 documents**. Quality over quantity.

Source	What to get
SEC EDGAR	Tesla, Panasonic 10-K supplier sections
Company sites	CATL, LG ESG/sustainability reports
Reuters/Bloomberg	EV battery supply chain articles
UN Comtrade	Lithium/cobalt trade flow summaries
Wikipedia	Overview pages for key companies

Save all as `.txt` files in a `/data/raw/` folder. Name them clearly: `tesla_10k_2023.txt`, `catl_esg_2023.txt`, etc.

Do not spend more than one day on this. 30 good documents is the ceiling.

Day 2 — Ontology & Graph Construction

[A] + [B]

[B] — Define the Ontology

Before extracting anything, lock down the schema. Node types:

- **Company** — *name, country, tier, type*
- **Material** — *name, category*
- **Country / Region** — *name, risk_score*
- **Facility** — *name, type, location*

Relationship types:

- `SUPPLIES_TO`, `SOURCES_FROM`
- `LOCATED_IN`, `OWNS`
- `PRODUCES`, `ALTERNATIVE_TO`

Write this as a Python dict that the extractor prompt references. Also build an **entity resolution map** — a lookup dict for known aliases:

```
aliases = {
    "Contemporary Amperex": "CATL",
    "LG Chem Energy": "LG Energy",
    "Republic of Congo": "DRC"
}
```

[A] — LLM Extraction Pipeline

Write a Python function that takes a document, sends it to the LLM with a zero-shot extraction prompt, and returns a list of triples.

Extraction prompt template:

```
You are a supply chain analyst. Extract entities
and relationships from the text below.
Return ONLY valid JSON | a list of triples:
{"subject": "...", "relation": "...",
 "object": "..."}

Entity types: Company, Material, Country,
Facility. Relation types: SUPPLIES_TO,
SOURCES_FROM, LOCATED_IN, OWNS, PRODUCES.

Text: {document}
```

Then write a Neo4j ingestion function:

```
MERGE (a:Company {name: $subj})
MERGE (b:Material {name: $obj})
MERGE (a)-[:SOURCES_FROM]->(b)
```

Run across all 30 documents. Expect 150–300 triples. Inspect the graph in Neo4j Browser and fix obvious errors manually.

Day 3 — GraphRAG Baseline & Agent Skeleton**[A] + [B]****[A] — GraphRAG Baseline**

Build a simple retrieve-then-generate function before adding the agent layer. Given a user query:

1. Extract key entities from the query (LLM call)
2. Run a Cypher query to fetch the relevant subgraph (2-hop neighborhood of those entities)
3. Serialize the subgraph as a list of triples
4. Pass triples + original query to LLM for synthesis

Test with simple queries first:

- “Who supplies lithium to CATL?”
- “Which countries does Tesla source cobalt from?”

Verify answers against what you know is in the graph. Fix retrieval gaps.

[B] — LangGraph Agent Skeleton

Build the agent state machine with 3 tools. Don’t make it smart yet — just get tool-calling working.

Tool 1 — Graph query:

```
def graph_query(cypher: str) -> str:
    result = neo4j_driver.run(cypher)
    return format_as_text(result)
```

Tool 2 — Web search:

```
def web_search(query: str) -> str:
    return tavily_client.search(query)
```

Tool 3 — Risk summarizer:

```
def risk_summarize(context: str) -> str:
    LLM call to produce structured
    risk report from context string
    ...
```

Wire into a LangGraph StateGraph with a ReAct loop. Test that the agent can call each tool independently before combining them.

Day 4 — Multi-Hop Agent Integration**[A+B]**

This is the most important day. Wire everything together and build the showcase query flow. Both people work on this together — one driving, one reviewing and testing.

The Killer Demo Query

Design the agent around this query:

“Is Tesla’s battery supply chain exposed to DRC political risk?”

Expected agent reasoning chain (6 steps):

1. **Graph query:** Find Tesla’s tier-1 battery cell suppliers → returns Panasonic, CATL
2. **Graph query:** Find what materials those suppliers source, and from where → returns cobalt from DRC
3. **Graph query:** Check if any other companies share this DRC cobalt dependency → concentration check
4. **Web search:** “DRC cobalt mining political risk 2024” → fetches current context
5. **Risk summarizer:** Combine graph facts + news context into structured risk assessment

6. Final output: Risk report with evidence trail, severity rating, and alternative sourcing options

Two Additional Showcase Queries

- “Which EV battery companies have the most concentrated lithium sourcing?” — tests global/aggregation reasoning across the graph
- “What alternative cobalt suppliers exist outside the DRC?” — tests the **ALTERNATIVE.TO** edges and counterfactual reasoning

Debugging Checklist

- Agent stops after 1 tool call → check the system prompt; instruct it to keep going until it has enough evidence
- Agent loops infinitely → add a **max_iterations=8** guard in LangGraph
- Graph query returns empty → the entity name in the query doesn't match the graph; check entity resolution
- LLM ignores tool results → make tool output more explicit; prefix with “**GRAPH RESULT:** ...”

Day 5 — Risk Scoring & Evaluation

[A] + [B]

[A] — Risk Scoring Layer

Add three lightweight risk metrics computed directly from the graph:

1. Concentration index (per material):

$$C_m = \frac{\text{supply from top country}}{\text{total supply nodes}} \times 100\%$$

2. Path count — number of distinct sourcing paths from Company X to material *m*. Low path count = high single-source risk.

```
MATCH path = (c:Company {name:$co})
-[:SUPPLIES_TO|SOURCES_FROM*1..3]->
(m:Material {name:$mat})
RETURN count(DISTINCT path) as paths
```

3. Geopolitical flag — hardcode a risk tier for each country node (High/Medium/Low) based on World Bank governance scores. DRC = High, Chile = Medium, Australia = Low.

Output a simple risk scorecard per company as a Python dict and as a printed table.

[B] — Evaluation

Build a small gold standard dataset manually — 15–20 supplier relationships you *know* are true from your source documents. Format:

```
gold = [
  {"s": "Tesla", "r": "SUPPLIES_TO",
   "o": "Panasonic"},
  {"s": "CATL", "r": "SOURCES_FROM",
   "o": "Lithium"},
  ...
]
```

Query the graph for each triple and measure:

$$\text{Precision} = \frac{|\text{correct triples in graph}|}{|\text{total triples in graph}|} \quad \text{Recall} = \frac{|\text{correct triples in graph}|}{|\text{gold triples}|}$$

Also evaluate 5 agent responses manually: score each on **faithfulness** (is every claim grounded in graph/search results?) and **completeness** (did it miss any relevant facts?). A simple 1–3 scale is enough for a class project.

Day 6 — Results & Writeup

[A] + [B]

[A] — Demo Outputs & Visualization

Produce the following artifacts for the report and demo:

1. **Graph visualization** — export the Neo4j graph as a NetworkX plot. Color nodes by type (Company = purple, Material = teal, Country = amber). Keep it to the 4 focal companies + their 2-hop neighborhood.
2. **Risk scorecards** — a table showing concentration index, path count, and geopolitical exposure per company.
3. **Agent trace** — print/log the full reasoning chain for the killer demo query. This is your most important result figure.
4. **Evaluation table** — precision/recall numbers + faithfulness scores.

Make sure the demo runs **end-to-end without errors** from a clean Python run. Practice it twice.

[B] — Report Structure

1. **Introduction** — supply chain visibility problem, why public data + LLMs, what you built
2. **Related work** — AlMahri et al. (2024), Microsoft GraphRAG (2024), one sentence each on LangGraph and ReAct
3. **Methodology** — ontology design, extraction pipeline, GraphRAG retrieval, agent architecture (include a simple pipeline diagram)
4. **Two-layer architecture (if extension done)** — public base layer vs private enrichment layer, access control design, agent tool routing
5. **Experiments** — graph statistics (node/edge counts), extraction evaluation (P/R), 3 agent query demos with reasoning traces
6. **Results & Discussion** — what the risk scores show, what the agent got right, where it failed
7. **Limitations** — public data ceiling (no volumes, no tier-3+, no private suppliers), graph staleness, hallucinations

8. **Future work** — federated KG across companies, continuous monitoring agent, counterfactual propagation
9. **Conclusion**

Extension (if time allows) — Private Data Integration Layer [optional] [A+B]

Only attempt this if Days 1–6 are fully working and stable. This extension transforms the system from a research prototype into a customizable enterprise tool — and is the strongest possible “future work made real” contribution for the report. Do not sacrifice the core demo for this.

The core idea

The public-data graph is a **base layer** — it captures what the world knows about the supply chain. A company’s private data is an **enrichment layer** — it adds what only the company knows: actual contracts, real volumes, internal supplier scorecards, operational flags. The agent reasons across both layers simultaneously, with private data taking precedence where it conflicts with public data.

What “private data” means here

For a class project, simulate private data with a small synthetic dataset. Create a CSV or JSON file representing what a fictional Company X would know internally:

```
private_suppliers.csv:
supplier, material, volume_mt,
contract_end, risk_flag
Panasonic, cells, 12000,
2026-12, low
CATL, cells, 8000,
2025-06, medium
Umicore, cobalt, 450,
2025-03, high
```

This is realistic — companies maintain exactly this kind of internal supplier register. The **risk_flag** and **contract_end** fields are the critical private signals that public data can never provide.

Architecture: two-layer graph

Extend the Neo4j schema with a **source** property on every node and edge:

- **source:** "public" — extracted from public documents
- **source:** "private" — ingested from company data

Add private-specific properties to relevant nodes:

- **volume_mt** — actual contracted volume
- **contract_end** — expiry date (urgency signal)
- **internal_risk_flag** — company’s own risk assessment
- **strategic_importance** — critical / substitutable

Ingestion is simple — read the CSV and run **MERGE** queries that add properties to existing nodes or create new private-only nodes.

Implementation checklist (half a day if the core pipeline is solid)

1. Create **private_data/suppliers.csv** with 8–10 synthetic rows (supplier, material, volume, contract_end, risk_flag)
2. Write **ingest_private.py** — reads CSV, runs **MERGE** queries tagging nodes with **source:"private"**
3. Add **private_query** tool to LangGraph agent
4. Update system prompt to use both tools and cite sources in output

Agent changes

Add one new tool: **private_query(cypher)** — identical to **graph_query** but filters to **source: "private"** nodes. The agent can now distinguish between public intelligence and internal knowledge, and explicitly flag in its output which claims come from which layer.

Update the agent system prompt to instruct it to:

- Check private data first for volume and contract details
- Cross-reference public risk signals against internal flags
- Escalate when a supplier has both a high public risk score *and* a short contract expiry

Example enriched query:

“Which of our contracted suppliers are in high-risk geographies and have contracts expiring within 6 months?”

This query is impossible with public data alone — it requires the internal contract dates. With the two-layer graph, the agent traverses both layers and surfaces an actionable procurement alert.

What this adds to the report

- A concrete demonstration that the system is not just a research toy but a deployable enterprise tool
- A clear architecture diagram showing public vs private layers (very visual, very presentable)
- A direct answer to the “but real companies have private data” question that any evaluator will ask
- Positions the work as a genuine contribution: public baseline + private enrichment = the full solution

5. Run the enriched demo query above and log the full reasoning trace
6. Add one slide / one report subsection: “Two-layer architecture: public + private data”

Code Cleanup

- Move all credentials to `.env`, never hardcoded
- Add a `README.md` with setup instructions (someone else should be able to run it)
- Clean up notebooks — remove failed cells, add mark-down headers
- Make sure the demo query runs in under 60 seconds end-to-end

Demo Flow (5–7 minutes)

1. Show the Neo4j graph visualization (30 sec) — “this is what we extracted from public data”
2. Run the killer query live (2 min) — show the agent reasoning step by step
3. Show the risk scorecard table (1 min)
4. Show the evaluation results (1 min)
5. *(If extension completed)* Show the two-layer demo query with private contract data (1 min)
6. Point to limitations + future work (1 min)

Common Last-Day Mistakes to Avoid

- Don’t add new features on day 7 — only stabilize what exists
- Don’t demo from a notebook with failed cells above — restart kernel and run top to bottom before presenting
- Have a backup: pre-run the agent query and save the output. If the live demo fails, show the saved output.
- Don’t apologize for scope — frame limitations as “future work,” not failures

Limitations & Future Work (Report Section)

Acknowledge Honestly

- **Public data ceiling:** tier-2+ relationships are sparse; no volume or financial dependency data; private suppliers invisible
- **Graph staleness:** supply chains change; a static graph goes stale within weeks
- **Hallucination risk:** LLM may generate plausible but ungrounded risk claims — mitigated by evidence trail but not eliminated
- **Entity resolution:** company name variants can create duplicate nodes; our alias map handles known cases only
- **Evaluation scale:** gold set of 15–20 triples is small; results are indicative, not statistically conclusive

Frame as Future Directions

- **Hybrid data architecture (partially implemented if extension done):** integrate firm-provided private data as an enrichment layer with access controls — establishes this work as the public-data *baseline*
- **Federated knowledge graphs:** multiple companies each contribute a private layer to a shared public graph without full mutual disclosure (privacy-preserving)
- **Continuous monitoring agent:** ingest news in real-time and update edge risk scores automatically
- **Counterfactual propagation:** agent simulates “what if Supplier Y fails?” by traversing and re-scoring the graph
- **Multi-industry benchmarking:** compare risk profiles across companies using the same public-data methodology — impossible with private-data approaches

Key papers to cite: AlMahri, Xu & Brintrup (2024) — *Enhancing Supply Chain Visibility with Knowledge Graphs and LLMs*, arXiv:2408.07705. Edge et al. (2024) — *From Local to Global: A Graph RAG Approach*, Microsoft Research. Yao et al. (2022) — *ReAct: Synergizing Reasoning and Acting in Language Models*, arXiv:2210.03629.